

CVM++

A Custom Interpreted Programming Language in C++ Project Report

Submitted by: Kushal Lakshya Tiwari
Roll No.: 240101051
Department: CSE
Institution: IIT Guwahati
Date: May 20, 2026

Abstract

CVM++ is a fully functional, interpreted programming language implemented in C++. The project demonstrates a complete language execution pipeline: a hand-written lexer and tokenizer, a recursive descent parser that constructs an Abstract Syntax Tree (AST), a bytecode compiler that translates the AST into intermediate instructions, and a stack-based virtual machine that executes those instructions. This report describes the design, architecture, implementation, and testing of each component.

Contents

1	Introduction	3
2	Objectives	3
3	System Design and Architecture	3
3.1	Lexer and Tokenizer	3
3.2	Recursive Descent Parser	4
3.3	Bytecode Compiler	4
3.4	Stack-Based Virtual Machine	4
4	Implementation	5
4.1	Lexer (<code>Lexer.h</code> / <code>Lexer.cpp</code>)	5
4.2	Parser (<code>Parser.h</code> / <code>parser.cpp</code>)	5
4.3	Bytecode Compiler (<code>chunk.h</code> / <code>ValueArray.h</code>)	6
4.4	Virtual Machine (<code>VirtualMachine.h</code>)	6
4.5	Supporting Modules	6
5	Bytecode Instruction Set	7
6	Testing and Results	7
7	Challenges and Solutions	7
7.1	Operator Precedence in the Parser	7
7.2	Jump Backpatching in the Compiler	8
7.3	Memory Management	8
7.4	Emitting bytecode	8
8	Conclusion	8
9	Future Work	8
	References	9
A	Project File Structure	10
B	Sample CVM++ Program	10
C	Build System (Makefile)	11

1 Introduction

Programming languages are the foundation of all software systems. While high-level languages abstract away low-level execution details, understanding how source code is transformed into executable instructions is essential for any serious computer science student.

CVM++ was developed to bridge this gap. By building each stage of a language interpreter from scratch in C++, this project provides a practical exploration of the following concepts:

- Lexical analysis and tokenization
- Syntax analysis via recursive descent parsing
- Intermediate representation using bytecode
- Execution through a stack-based virtual machine

CVM++ is not merely a theoretical exercise — it is a working language implementation capable of parsing and executing programs written in its custom `.cvm` syntax.

2 Objectives

The primary objectives of this project are:

1. Design and implement a working programming language from scratch in C++.
2. Build a hand-written lexer and tokenizer to process raw source text.
3. Implement a recursive descent parser to produce an Abstract Syntax Tree (AST).
4. Develop a bytecode compiler that translates the AST into executable instructions.
5. Create a stack-based virtual machine to execute the generated bytecode.
6. Ensure the system is modular, readable, and extensible.

3 System Design and Architecture

CVM++ is structured as a sequential pipeline. Source code passes through each stage before final execution:

Source Code (`.cvm`) → Lexer / Tokenizer → Parser (AST) → Bytecode
Compiler → Stack-Based VM

Figure 1: CVM++ Execution Pipeline

3.1 Lexer and Tokenizer

The lexer reads the source file character by character and groups characters into tokens. Each token carries a type (defined in `Tokens.h`) and its literal value. The tokenizer handles whitespace, comments, literals, identifiers, keywords, operators etc.

3.2 Recursive Descent Parser

The parser (implemented in `Parser.h` / `parser.cpp`) consumes the token stream and applies grammar rules using mutually recursive functions. The output is an Abstract Syntax Tree (AST), whose node types are defined in `ASTNodes.h`. This approach naturally encodes operator precedence through the function call hierarchy.

The current Grammar used for this project:

```

program → statement* EOF ;
statement → IfStmt | WhileStmt | PrintStmt | InputStmt | ExprStmt ;
IfStmt → "if" "("expression")" statement ";" ;
WhileStmt → "while" "("expression")" statement ";" ;
InputStmt → "input" expression ";" ;
PrintStmt → "print" expression ";" ;
ExprStmt → expression ";" ;

```

```

expression → equality ;
assignment → identifier "=" assignment ;
equality → comparison ( "("==" comparison )" )* ;
comparison → term ( ( "<" | ">" ) term )* ;
term → factor ( ( "-" | "+" ) factor )* ;
factor → unary ( ( "/" | "*" ) unary )* ;
unary → ( "!" ) unary | primary ;
primary → number | boolean | identifier | "(" expression ")" ;

```

3.3 Bytecode Compiler

The bytecode compiler performs a tree-walk over the AST and emits a sequence of instructions into a **Chunk** (defined in `chunk.h`) using the **Visitor Pattern**. A **ValueArray** (`ValueArray.h`) stores the constant pool. Jump targets for control flow are resolved via backpatching after the target bytecode is emitted.

3.4 Stack-Based Virtual Machine

The virtual machine (`VirtualMachine.h`) maintains an operand stack, an instruction pointer, hash table for storing global variables. It fetches each opcode, decodes it, and executes the corresponding operation. Variable storage is backed by a **HashTable** (`HashTable.h`).

Table 1: CVM++ Pipeline Stages and Key Files

Stage	Component	Key Files
1	Lexer / Tokenizer	<code>Lexer.h</code> , <code>Lexer.cpp</code> , <code>Tokens.h</code>
2	Recursive Descent Parser	<code>Parser.h</code> , <code>parser.cpp</code> , <code>ASTNodes.h</code>
3	Bytecode Compiler	<code>chunk.h</code> , <code>ValueArray.h</code> , <code>Compiler.h</code>
4	Stack-Based Virtual Machine	<code>VirtualMachine.h</code> , <code>HashTable.h</code> , <code>memory.h</code>

4 Implementation

4.1 Lexer (Lexer.h / Lexer.cpp)

The lexer scans source text character by character, identifies token boundaries, and produces a flat list of tokens. Each token carries its type enum class (`TokenType` from `Tokens.h`), its raw lexeme, and its line number for error reporting.

```

1 struct Token{
2     TokenType type = TokenType::Default;
3     std::string val;
4     int line;
5     Token(TokenType type, int line) : type(type), val(""), line
      (line){}
6     Token(TokenType type, std::string val, int line) : type(
      type), val(val), line(line){}
7 };

```

Listing 1: Token structure (Tokens.h)

4.2 Parser (Parser.h / parser.cpp)

The recursive descent parser converts the token stream into an AST. Each grammar rule corresponds to a C++ function. These functions call each other recursively, naturally encoding precedence. Node types (expression, statement, declaration, etc.) are defined in `ASTNodes.h`.

```

1 std::unique_ptr<ExpressionAST> Parser::term()
2 {
3     auto expr = factor();
4     while (match({TokenType::Plus, TokenType::Minus}))
5     {
6         Token op = previous();
7         expr = std::make_unique<Binary>(op, std::move(expr),
8             factor());
9     }
10    return expr;
11 }
12 std::unique_ptr<ExpressionAST> Parser::factor()
13 {
14     auto expr = unary();
15     while (match({TokenType::Mul, TokenType::Div}))
16     {
17         Token op = previous();
18         auto right = unary();
19         expr = std::make_unique<Binary>(op, std::move(expr),
20             std::move(right));
21     }
22    return expr;
23 }

```

Listing 2: Example recursive descent function skeleton

4.3 Bytecode Compiler (`chunk.h` / `ValueArray.h`)

The compiler performs a post-order traversal of the AST and emits opcodes into a `Chunk`. Constants (numbers, strings) are stored in a `ValueArray`. Control flow (if/else, while) uses backpatching to resolve jump targets.

```

1  class Chunk{
2  public:
3      int capacity;
4      int count;
5      int *lines;
6      uint8_t *code;
7      ValueArray constants;
8      void initChunk(){

```

Listing 3: Chunk structure (`chunk.h`)

4.4 Virtual Machine (`VirtualMachine.h`)

The VM maintains an operand stack and an instruction pointer. It fetches each opcode in a loop and dispatches to the appropriate handler. Local and global variables are stored in a `HashTable`.

```

1  InterpretResult run()
2      {
3      // READ_BYTE, READ_CONSTANT, BINARY_OP are defined
4      // first.
5
6      // Main execution below
7      for (;;)
8      {
9          Opcode instruction;
10         switch (instruction = (Opcode)READ_BYTE())
11         {
12             case (Opcode::Op_Return):
13             {
14                 return InterpretResult::INTERPRET_OK;
15                 break;
16             }
17             case Opcode::Op_Constant:
18             {
19                 Value rconstant = READ_CONSTANT();
20                 push(rconstant);
21                 break;
22             }
23         }
24     }
25 }

```

Listing 4: VM execution loop skeleton

4.5 Supporting Modules

- `HashTable.h` — Open-addressing hash map for global variable storage.
- `memory.h` — Centralized allocation and dynamic array growth utilities.

- `commons.h` — Common type aliases.
- `error.h` — Error reporting with line number and descriptive messages. (Not completed yet)

5 Bytecode Instruction Set

Table 2: CVM++ Opcode Reference

Opcode	Description
OP_CONSTANT	Push a constant from the pool onto the stack
OP_ADD	Pop two values, push their sum
OP_SUB	Pop two values, push their difference
OP_MUL	Pop two values, push their product
OP_DIV	Pop two values, push their quotient
OP_EQUALITY	Push true if top two values are equal
OP_LESSER	Push true if second value < top value
OP_GREATER	Push true if second value > top value
OP_NOT	Logical NOT of the top value
OP_JUMP_IF_FALSE	Jump if top of stack is false/zero
OP_JUMP	Jump backwards (for while loops)
OP_GET_GLOBAL	Push value of a global variable
OP_SET_GLOBAL	Assign value to a global variable
OP_PRINT	Pop and print the top value
OP_INPUT	Take input directly from user and push it onto the stack
OP_RETURN	End VM execution

6 Testing and Results

The following test cases were used to validate the CVM++ pipeline end-to-end:

Table 3: Test Cases and Results

Test Case	Description	Result
Hello World	Print a string literal	Pass
Arithmetic	Basic +, -, *, / operations	Pass
Variables	Variable declaration and assignment	Pass
Conditionals	if branching	Pass
Loops	while loop with counter	Pass

7 Challenges and Solutions

7.1 Operator Precedence in the Parser

Challenge: Correctly handling operator precedence without explicit precedence tables.
Solution: Modeled precedence through the grammar hierarchy in the recursive descent

parser. Lower-precedence rules call higher-precedence ones, naturally producing correct evaluation order.

7.2 Jump Backpatching in the Compiler

Challenge: When compiling `if/else` and `while` constructs, jump target offsets are unknown at emit time.

Solution: Emitted placeholder jump offsets and backpatched them after generating the target bytecode.

7.3 Memory Management

Challenge: Managing dynamic memory safely in C++ without leaks or dangling pointers.

Solution: Centralized allocation through `memory.h` using `GROW_ARRAY` and `FREE_ARRAY` macros for consistent, traceable allocations.

7.4 Emitting bytecode

Challenge: Clean, Readable and Manageable code for bytecode generation.

Solution: Use a **Visitor pattern** which makes a common accept function for all nodes. All the emit functions are inside `Compiler.h` which can be easily edited and managed.

8 Conclusion

CVM++ successfully demonstrates a complete language implementation pipeline in C++. Starting from raw source text, the system performs lexical analysis, recursive descent parsing, bytecode compilation, and stack-based execution in a clean, modular architecture.

Building CVM++ provided deep practical insight into how programming languages work at each stage of execution — from character streams and token recognition, through syntax analysis and tree construction, to bytecode generation and runtime execution. The project serves as both a functional interpreter and an educational reference for language implementation in C++.

9 Future Work

- Add support for all other unary and binary operators
- Add support for more data types: string, arrays, dictionaries, and user-defined structs.
- Implement a mark-and-sweep garbage collector for automatic memory management.
- Add functions, blocks and local variables
- Develop a REPL (Read-Eval-Print Loop) for interactive use.
- Add syntax error detection in the parser and runtime error detection in the VM for better diagnostics.

References

1. Nystrom, R. (2021). *Crafting Interpreters*. Available at: <https://craftinginterpreters.com>

A Project File Structure

```

CVM++/
|-- include/
|   |-- ASTNodes.h      AST node type definitions
|   |-- chunk.h         Bytecode chunk (instructions + constant pool)
|   |-- commons.h       Shared macros and type aliases
|   |-- error.h         Error handling and reporting
|   |-- HashTable.h     Hash map for variable storage
|   |-- Lexer.h         Lexer class interface
|   |-- memory.h        Memory allocation utilities
|   |-- Parser.h        Parser class interface
|   |-- Tokens.h        Token type enum and struct
|   |-- ValueArray.h    Dynamic array for VM values
|   |-- VirtualMachine.h Stack-based VM interface
|   |-- Compiler.h      The compiler which generates bytecode. Visitor pattern is
|   |-- Printer.h       Printer for the AST. Visitor pattern is used.
|
|-- src/
|   |-- Lexer.cpp        Lexer / tokenizer implementation
|   |-- parser.cpp       Recursive descent parser
|   |-- main.cpp         Entry point
|
|-- build/               Compiled .o object files
|-- programs/
|   |-- program_code.cvm Sample CVM++ program
|
|-- makefile             Build system
|-- main.exe            Compiled executable
|-- README.txt          Project overview
|-- instructions.txt     Build and usage guide

```

B Sample CVM++ Program

```

1 print "Hello from CVM !";
2
3 x = 10;
4 y = 20;
5 print (x + y);

```

Listing 5: programs/program_code.cvm

```

1 x = 10;
2 while(x > 5) x = x-2;
3 print x;

```

Listing 6: programs/program_code.cvm

```

1 x = input "Enter Number : ";
2

```

```
3 if(x > 10) print "x is more than 10 \n";
4 if(x < 10) print "x is less than 10 \n";
5 if (x == 10) print "x is equal to 10 \n"
6
7 print "Programme Complete.";
```

Listing 7: programs/program_code.cvm

C Build System (Makefile)

```
1 CXX = g++
2 CXXFLAGS = -std=c++17 -Wall -Wextra -g -I include
3
4 SRC_DIR = src
5 BUILD_DIR = build
6
7 SRCS = $(wildcard $(SRC_DIR)/*.cpp)
8 OBJS = $(patsubst $(SRC_DIR)/%.cpp, $(BUILD_DIR)/%.o, $(SRCS))
9 TARGET = main
10
11 all: $(BUILD_DIR) $(TARGET)
12
13 $(BUILD_DIR):
14     mkdir -p $(BUILD_DIR)
15
16 $(TARGET): $(OBJS)
17     $(CXX) $(CXXFLAGS) -o $@ $^
18
19 $(BUILD_DIR)/%.o: $(SRC_DIR)/%.cpp
20     $(CXX) $(CXXFLAGS) -c $< -o $@
21
22 run: all
23     ./$(TARGET)
24
25 clean:
26     rm -rf $(BUILD_DIR) $(TARGET)
27
28 .PHONY: all clean run
```

Listing 8: makefile